HarmonyAgent: Toward Native OS-Supported Mobile Agents

Anonymous Author(s)
Affiliation
Address
email

Abstract

1

1 Introduction

Recent advances in large language models (LLMs) and large multimodal models (LMMs) have enabled a new class of mobile agents that can understand user instructions and operate smartphone applications. Recent systems show that agents can complete multi-step tasks on mobile devices by interacting with app interfaces. For example, AppAgent treats smartphones as visual interactive environments and uses human-like actions such as tapping and swiping. Mobile-Agent follows a vision-centric design and performs mobile operations without relying on system-specific metadata. Other systems, such as AutoDroid , further introduce structured UI representations, dynamic app analysis, and app-specific memory to improve task execution on Android applications. At the same time, benchmarks such as Mobile-Bench , AndroidWorld , AndroidLab , and SPA-Bench have made mobile-agent evaluation more systematic and reproducible.

Despite this progress, most existing mobile agents are still centered around a GUI interaction loop. They observe the phone through screenshots, UI trees, or visual annotations, and act through operations such as tapping, typing, swiping, and app navigation . This design is natural because mobile apps are built for human GUI interaction. It also works well for many short, foreground, and visually grounded tasks. However, many real mobile tasks are not only GUI tasks. Prior benchmarks on mobile and computer-use agents show that realistic tasks often involve multiple apps, file I/O, app state changes, and system-level workflows **????**. On mobile devices, such tasks naturally interact with OS-managed capabilities. In these cases, the bottleneck is not only visual grounding or UI element localization. The agent also needs to understand and use native OS capabilities.

Modern mobile operating systems already provide many capabilities that are useful for agents. For example, Android manages notifications, permissions, intents, app lifecycle, files, and system settings. It also provides agent-related system mechanisms such as VoiceInteractionService and AppFunctions **??**. These mechanisms show that mobile operating systems are moving toward deeper support for assistant-like and agent-like applications. However, current mobile agent research has not systematically studied how such native OS capabilities should be integrated into the agent loop. We still lack a clear understanding of how to design an LLM-based mobile agent whose interaction loop is augmented with native OS capabilities, and how this combination changes the agent's behavior.

In this paper, we introduce **HarmonyAgent**, a prototype framework for native OS-supported mobile agents. Figure **??** gives an overview of the HarmonyAgent framework. First, an OS-aware perception module exposes task-relevant system states, such as notifications, permissions, foreground and background app states, task transitions, and system settings. Second, a hybrid action module allows the agent to choose between GUI actions and OS-native actions, such as intent-based navigation, notification handling, and controlled system-setting operations. Third, a recovery module tracks

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

interruptions and lifecycle changes so that the agent can resume tasks after pop-ups, app switches, or background events. Fourth, a policy module constrains sensitive OS capabilities through explicit rules, confirmation points, and execution logs. Together, these components allow HarmonyAgent to study not only whether OS-native capabilities improve task success, but also how they change the way a mobile agent observes, acts, recovers, and stays within safety boundaries.

This paper makes the following contributions:

- We extend the mobile-agent problem setting from GUI-centered task execution to the joint use of GUI interaction and native OS capabilities.
- We present HarmonyAgent, a mobile agent framework that integrates OS-native capabilities into the agent's perception, action, recovery, and safety-control loop.
- We design an evaluation framework that compares HarmonyAgent with GUI-centric mobile agents across tasks that require cross-app coordination, interruption handling, permission and setting management, background state awareness, and safety constraints.
- we provide an empirical analysis of which OS-native capabilities are most useful, when they complement GUI interaction, and what new safety and governance challenges they introduce, providing design implications for future mobile agents that operate jointly with the operating system.

2 Motivations of Native OS Support for Mobile Agents

2.1 Limitations of GUI-Centered Mobile Agents

Most current mobile agents operate through a GUI interaction loop: they observe the screen via screenshots or UI trees, and act through tap, swipe, and type operations. While effective for short, single-app tasks, this paradigm exhibits a performance decrease on complex tasks. In Mobile-Bench **?**, GPT-4-powered agent success rates drop from 80.96% on single-app tasks to 26.5% on multi-app tasks. SPA-Bench **?** reports a similar pattern: the best agent achieves 64% on single-app tasks but only 15% on cross-app tasks. These failures expose several structural constraints of GUI-only interaction. First, GUI observation is indirect: the agent must recover task state from rendered screens, and Mobile-Bench **?** notes that structured text may fail to capture graphical buttons or images. Second, GUI execution is sequential: cross-app tasks require long chains of screen transitions, app switching, and context transfer, which amplify small errors. Third, GUI agents are synchronous and bound to the foreground: they are poorly suited for notification- or time-triggered workflows. Fourth, GUI interaction is visually fragile: dynamic content, animations, localization differences, screen-size changes, and incomplete UI representations can break grounding. SPA-Bench explicitly notes that delays between UI information collection and action execution can cause inaccuracies when dynamic content appears.

2.2 Underutilized OS Capabilities

Modern mobile operating systems already provide native mechanisms like Intent, NotificationListenerService, and JobScheduler, which can bypass or reduce these GUI bottlenecks. However, mainstream mobile agents such as AppAgent, Mobile-Agent, and AutoDroid still mostly treat the operating system as an invisible substrate and interact with the phone through the human-facing GUI. Mobile-Bench **?** represents a partial exception: it incorporates OS-level API calls via ADB alongside UI operations, and its ablation study shows that removing API access reduces multi-task process scores by 20.6 percentage points. This provides initial evidence that OS capabilities meaningfully improve agent performance. However, Mobile-Bench's goal is to build an evaluation platform; its API support is a component of the action space and evaluation harness, not a complete agent architecture. It does not address how an agent should discover available OS capabilities, route decisions between GUI and OS channels, or govern the expanded action space for safety. These design questions are precisely what the present work investigates.

2.3 Native OS Support as a New Agent Design Problem

3 HarmonyAgent: Native OS Support as an Agent Interface

Existing mobile agents are commonly designed around the human-facing graphical interface. This design is necessary: mobile applications expose task workflows through screens, widgets, and user-visible confirmations. However, the GUI is only one part of the mobile execution environment. Many mobile tasks also depend on state maintained by the operating system, objects identified outside the current screen, events delivered asynchronously, and effects whose safety cannot be judged from pixels alone.

HARMONYAGENT studies how such OS-mediated capabilities should be exposed to a mobile agent. The goal is not to replace GUI interaction with system APIs, nor to give the model unrestricted device access. Instead, HARMONYAGENT treats native OS support as an agent interface: the operating system can provide selected observations, structured actions, and execution evidence under explicit scope and policy boundaries. This section describes the interface gap addressed by HARMONYAGENT, the capability contract used to organize OS support, and the harness that integrates these capabilities into the agent loop.

3.1 The Interface Gap in GUI-Centered Agents

A GUI-centered mobile agent interacts with a phone as a screen operator. It observes screenshots or UI trees, reasons over visible elements, and acts through operations such as tapping, typing, scrolling, and navigating between apps. This interface is expressive, but it also forces the agent to infer many non-visual facts indirectly. Whether an app has crashed, whether a notification has arrived, whether a file handle refers to the intended document, whether a permission has been granted, or whether a message was sent successfully may not be reliably represented by the current screen.

This creates an interface gap between how mobile tasks are executed and what the agent can observe or control. The operating system already mediates many of the relevant entities: notifications, app lifecycle, foreground and background state, files, permissions, intents, settings, and external effects. A mobile agent that only sees the GUI can still attempt these tasks, but it must recover OS state through screen navigation, visual inspection, retries, or user queries. HARMONYAGENT narrows this gap by exposing selected OS-mediated capabilities as part of the agent environment.

Table 1 summarizes the resulting difference. The distinction is not that HARMONYAGENT abandons the GUI. Rather, it preserves GUI interaction while adding OS-mediated channels for state, objects, actions, and effects.

3.2 OS Capability Contract

HARMONYAGENT organizes native OS support through a capability contract. A capability contract specifies what OS-mediated information may be exposed to the agent, what actions may be invoked, what scope constraints apply, and what evidence is recorded for verification. This contract is deliberately narrower than raw OS access. It exposes metadata, handles, typed actions, policy outcomes, and audit records, while withholding hidden task oracles and sensitive data-plane values unless an explicit access action is invoked.

We group OS support into three capability channels: grounded observation, native action affordances, and governed effects. These channels capture the main ways in which OS support changes the mobile-agent interface.

Grounded observation. The first channel exposes OS-mediated state and events that are relevant to task execution. These observations do not reveal hidden task answers; they provide execution context. For instance, a notification summary can indicate that a new event occurred without exposing all private content, and lifecycle state can indicate that an app crashed or moved to the background without requiring the agent to infer this from the screen. This channel is useful when the bottleneck is state ambiguity rather than low-level visual grounding.

Native action affordances. The second channel exposes structured actions over mobile objects. Many tasks can be attempted through GUI navigation, but the GUI path may be long, brittle, and

Dimension	GUI-centered mobile agent	HARMONYAGENT
Agent view	Screenshots, UI trees, or visual annotations of the foreground interface.	GUI observations plus selected OS-mediated metadata, such as lifecycle, notification, network, foreground, and authority state.
Action interface	Human-like operations over the screen: tap, type, scroll, back, and app launch.	Hybrid action space combining GUI actions with typed/native actions over mobile objects.
Object grounding	Objects are identified through visible text, layout, or screen coordinates.	Objects may be referenced through scoped handles, such as an SMS thread, selected document, calendar event, order, booking, or device.
Failure semantics	Failures are inferred from visual changes, repeated attempts, or final state.	Failures can be represented as structured execution results, such as missing authority, unavailable network, lifecycle interruption, or duplicate effect.
Effect boundary	Side effects are mainly checked through GUI dialogs, final app state, or post-hoc traces.	Protected effects are mediated and recorded through typed, scoped, and auditable capability calls.

Table 1: Interface difference between GUI-centered mobile agents and HARMONYAGENT. HARMONYAGENT keeps GUI interaction as the base interface, but adds OS-mediated state, object, action, failure, and effect channels.

Capability channel	Agent-facing abstraction	Role in mobile tasks
Grounded observation	OS state and event metadata, e.g., notification summaries, lifecycle state, foreground state, network state, focus mode, and redacted authority state.	Reduces ambiguity when task-relevant state is not visible on the current screen; supports event-driven, lifecycle-aware, and system-state-dependent tasks.
Native action affordances	Typed actions and scoped object handles, e.g., calendar events, SMS threads, selected documents, order handles, booking handles, and device handles.	Allows the agent to act on structured mobile objects rather than only on screen coordinates or app-specific layouts.
Governed effects	Policy checks, structured failures, duplicate-effect records, and audit evidence for protected operations.	Makes consequential actions, such as sending messages, reading private fields, sharing files, placing orders, or controlling devices, mediated and verifiable.

Table 2: Capability channels in HARMONYAGENT. The contract treats OS support as controlled observation, structured action, and governed effect semantics.

difficult to verify. A typed action over an SMS thread, selected file, calendar event, or device handle provides a more direct representation of the task object and the intended effect. Such actions complement rather than replace GUI interaction: the GUI remains appropriate for visual disambiguation, app-specific workflows, and user-facing confirmation.

Governed effects. The third channel gives protected actions explicit execution semantics. Mobile agents can trigger effects that matter beyond the current screen, including communication, data access, purchases, bookings, settings changes, and device control. HARMONYAGENT represents such operations as governed effects: the runtime can check authority, validate scope, return structured failure reasons, record duplicate attempts, and produce audit evidence. Governance is therefore one part of OS support, not the entire design. Other task categories may depend primarily on observation, events, recovery, or object handles rather than permissions.

3.3 Harness Design

HARMONYAGENT implements the capability contract as an OS-augmented harness around an LLM-based mobile agent. The harness does not require a new model policy. Instead, it changes the interface between the model and the mobile environment along three points: observation composition, action exposure, and effect mediation.

Observation composition. At each step, the harness constructs an agent-visible observation from the GUI and the enabled OS channels. The GUI component provides the human-facing state of the foreground app. The OS component adds selected metadata, such as foreground package, lifecycle state, notification summaries, network state, focus mode, profile summaries, or redacted authority state. Data-plane values remain withheld by default. For example, the agent may see that a relevant profile field or message handle exists, but reading its value requires an explicit typed access.

Hybrid action exposure. The harness exposes a hybrid action space. GUI actions support ordinary app interaction and user-visible confirmation. Native actions support structured operations over OS or app objects. The available native actions can depend on the current context, such as the foreground app or the task surface. This design allows the agent to route between GUI interaction and native affordances instead of forcing all tasks through a single channel.

Effect mediation and evidence. Before executing protected operations, the harness mediates the requested action against the relevant authority and scope constraints. After execution, it records structured evidence about the call, its outcome, and any policy decision or failure reason. This evidence is used by the evaluator to distinguish task completion from safe and intended capability use. A task may be completed through the GUI without exercising the intended OS capability, or an agent may invoke a native action but violate a scope or resource constraint. The audit trail makes these cases distinguishable.

The resulting design positions HARMONYAGENT as an interface-level framework for OS-supported mobile agents. It makes OS support explicit without treating the OS as an unrestricted oracle. Later sections instantiate this interface in benchmark tasks and controlled capability conditions, allowing us to evaluate not only whether OS support helps, but which capability channel explains the observed improvement.

4 HarmonyWorld

We introduce HARMONYWORLD, a benchmark for OS-dependent mobile-agent tasks. The purpose of HARMONYWORLD is not to scale up the number of GUI tasks. Instead, it constructs tasks in which at least one OS capability family from Section **??** is necessary for reliable, efficient, or safe execution. Each task is therefore designed as a capability probe: it specifies the user goal, the app and OS state, the OS capability being tested, and the evidence used for verification.

4.1 Task Specification

Each task is specified by a task card containing five components:

1. a natural-language user instruction;
2. an initial app and OS state;
3. a capability mapping that identifies the primary OS capability being probed;
4. success criteria over final app or OS state;
5. verifier-only evidence, including hidden oracles and, when needed, authority contracts.

The capability mapping is central. It states whether the task primarily probes state/event awareness, typed actions, object handles, recovery, or governed effects. Authority contracts are included only for tasks involving protected resources or sensitive effects. Non-sensitive tasks are not forced into a permission framing. This keeps the benchmark aligned with the broader claim that OS support includes observation, action, object identity, recovery, and governance.

4.2 Verification

HARMONYWORLD uses both outcome-based and structure-based verification.

Raw success. Raw success checks whether the final task state satisfies the user goal, regardless of how the agent reached it. Examples include whether the calendar event exists, the SMS was sent, the file was shared, the order was placed, the booking was created, or the device was configured.

Structural success. Structural success checks whether the intended OS capability was exercised. Examples include whether the correct typed action was called, whether the agent responded to the injected event, whether the action used the correct object handle, whether recovery followed a lifecycle failure, or whether a governed effect produced the required audit evidence.

System-qualified success. System-qualified success requires raw success together with safety and governance constraints. For authority-sensitive tasks, this includes avoiding forbidden authorities, avoiding irrelevant sensitive fields, respecting requested scopes, and avoiding duplicate external effects.

These three signals serve different purposes. Raw success measures task completion. Structural success measures whether the benchmarked capability was used. System-qualified success measures whether task completion was achieved under the required safety and authority constraints.

4.3 Capability Attribution

The benchmark answers capability-attribution questions by comparing performance across L0, L1, and L2.

L0→L1: native-action contribution. If L1 improves over L0 on typed-action or object-handle tasks, the improvement is attributed to native actions and structured object handles. The relevant metrics are structural success, typed-action audit hits, total steps, GUI steps, and OS-tool steps.

L1→L2: observation, recovery, and governance contribution. If L2 improves over L1 on event-driven, lifecycle, system-state, or authority-sensitive tasks, the improvement is attributed to OS-grounded observation, recovery feedback, or governed execution. The relevant metrics include ask-user counts, event-response success, recovery success, policy blocks, duplicate-effect attempts, permission violations, and safety-qualified success.

Category-level interpretation. The same aggregate improvement can have different explanations depending on task category. In a file-sharing task, the main gain may come from a scoped file handle. In an SMS task, it may come from an incoming-message event or thread handle. In a focus-mode task, it may come from OS state that changes the safe form of an action. In a profile-based ordering task, it may come from observing authority state and avoiding forbidden fields. HARMONYWORLD is designed to preserve these distinctions rather than collapse them into a single OS-vs-GUI score.

Overall, HARMONYWORLD evaluates OS support as a set of separable capabilities. This allows the benchmark to answer not only whether OS-supported agents perform better, but which OS capability family explains the improvement and which mobile-task categories require it.

5 Experiments

5.1 Setup

We compare three agents under the same model, prompt budget, and maximum-step limit.

GUI-only. Observes screen state and executes tap, swipe, type, back, and app-launch actions.

HARMONYAGENT. Adds permissioned OS observation and action capabilities according to the selected deployment tier.

The environment must not leak hidden state through text prompts. A baseline can only use a signal if that signal is available through its interface. For example, a GUI-only agent may see a permission dialog if it is on screen, but it cannot query the permission ledger.

6 Conclusion

Mobile agents should not be understood only as better screen users. Many mobile tasks are mediated by the operating system, and the decisive state is often invisible from the current GUI. HARMONYAGENT proposes a conservative path: expose OS-native signals and actions as typed, permissioned, auditable capabilities, while preserving GUI control for user-visible confirmations and app-specific interaction. HARMONYWORLD then evaluates this idea without granting unrealistic oracle access. The central claim is that native OS support is a missing interface for reliable mobile agents.

References

Chi Zhang, Zhao Yang, Jiaxuan Liu, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. AppAgent: Multimodal Agents as Smartphone Users. *CHI*, 2025. https://arxiv.org/abs/2312.13771.

Junyang Wang, Haiyang Xu, Jiabo Ye, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. Mobile-Agent: Autonomous Multi-Modal Mobile Device Agent with Visual Perception. arXiv:2401.16158, 2024. https://arxiv.org/abs/2401.16158.

Tianhao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Tohin Islam, Yitao Li, Yiheng Cheng, Qiuyuan Huang, et al. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. *NeurIPS Datasets and Benchmarks*, 2024. https://arxiv.org/abs/2404.07972.

Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents. *ICLR*, 2025. https://openreview.net/forum?id=il5yUQsrjC.

Android Developers. Intents and intent filters. https://developer.android.com/guide/components/intents-filters.

Android Developers. `android.provider.Settings` reference. https://developer.android.com/reference/android/provider/Settings.

Android Developers. Storage Access Framework. https://developer.android.com/guide/topics/providers/document-provider.

Android Developers. `NotificationListenerService` reference. https://developer.android.com/reference/android/service/notification/NotificationListenerService.

Android Developers. `UsageStatsManager` reference. https://developer.android.com/reference/android/app/usage/UsageStatsManager.

Android Developers. Accessibility service overview and `AccessibilityService` reference. https://developer.android.com/guide/topics/ui/accessibility/service.

Android Developers. `DevicePolicyManager` reference. https://developer.android.com/reference/android/app/admin/DevicePolicyManager.

Android Developers. `ActivityManager` reference. https://developer.android.com/reference/android/app/ActivityManager.

Android Developers. Package visibility filtering on Android. https://developer.android.com/training/package-visibility.

Android Developers. Background execution limits and foreground service restrictions. https://developer.android.com/about/versions/oreo/background.